Predicting Code Coverage without Execution

Michele Tufano, Shubham Chandel, Anisha Agarwal, Neel Sundaresan, Colin Clement
Microsoft
Redmond, WA, USA
{mitufano, schandel, anisagarwal, neels, coclement}@microsoft.com

arXiv:2307.13383v1 [cs.SE] 25 Jul 2023

Abstract

Code coverage is a widely used metric for quantifying the extent to which program elements, such as statements or branches, are executed during testing. Calculating code coverage is resource-intensive, requiring code building and execution with additional overhead for the instrumentation. Furthermore, computing coverage of any snippet of code requires the whole program context. Using Machine Learning to amortize this expensive process could lower the cost of code coverage by requiring only the source code context, and the task of code coverage prediction can be a novel benchmark for judging the ability of models to understand code. We propose a novel benchmark task called Code Coverage Prediction for Large Language Models (LLMs). We formalize this task to evaluate the capability of LLMs in understanding code execution by determining which lines of a method are executed by a given test case and inputs. We curate and release a dataset we call COVERAGEEVAL by executing tests and code from the HumanEval dataset and collecting code coverage information. We report the performance of four state-of-the-art LLMs used for code-related tasks, including OpenAI's GPT-4 and GPT-3.5-Turbo, Google's BARD, and Anthropic's Claude, on the Code Coverage Prediction task. Finally, we argue that code coverage as a metric and pre-training data source are valuable for overall LLM performance on software engineering tasks.

1 Introduction

Software testing is an essential part of the software life-cycle which aims at detecting bugs in a program prior to shipping new versions. Code coverage is a widely used metric which estimates the quality of testing, providing some confidence that the system will operate conforming to the specified requirements. Several standards require a specific level of code coverage for software systems before they are allowed to be deployed.

Focal Method $\{m\}$

```
public String foo(int x){
    if(x == 0){
        return "zero";
    } else if(x > 0){
        return "positive";
    } else {
        return "negative";
    }
    return "impossible";}
```

Test Case $\{t\}$

```
public void testFoo() {
    String res = foo(2);
    Assert.isEqual("positive", res);}
```

Coverage-Annotated Method $\{\mathrm{cov}(m,t)\}$

```
> public String foo(int x){
>     if(x == 0){
!         return "zero";
>     } else if(x > 0){
>         return "positive";
!     } else {
!         return "negative";
!     }
-     return "impossible";}
```

Figure 1: Given a focal method m, that is a method under test, and a test case t covering that method, the code coverage obtained by t on m can be represented as the coverage-annotated method $\mathrm{cov}(m,t)$, where > represents executed statements, ! represents statements not executed, and - represents unreachable code.

For example, coverage is one of the metrics considered by the Federal Aviation Administration (FAA) for safety certification of avionic equipment, as documented in DO-178B (Johnson, 1998) and DO-178C (Rierson, 2017). Test coverage is also a requirement in the automotive safety standard ISO 26262 Road Vehicles - Functional Safety (Palin et al., 2011).

Given a focal method m, which is executed *directly* by the test case t, code coverage measures the number of statements that have been executed (*i.e.,* covered) by the test t. Figure 1 shows an example of a focal method m (method under test) tested by t. The coverage obtained by t on m

is represented in the coverage-annotated method $\mathrm{cov}(m, t)$, where executed statements are marked with > while missed (*i.e.,* uncovered statements) with ! and unreachable code (*i.e.,* dead code) with -. From this representation, several quantitative coverage metrics can be computed, such as functional, statement, branch, and path coverage.

Code coverage is computed by instrumenting the code and running the test suite while monitoring the code execution. This process is expensive, since it requires building and executing code, especially for large software projects or when code coverage is computed multiple times. Additionally, it is not possible to measure code coverage for a snippet of code without the availability of the entire program which contains the given snippet. This situation happens when only partial code is available, for example within a commit log/diff, or when only partial code is transmitted to a server, for security and/or networking reasons.

While Large Language Models (LLMs) have gained prominence in code-related tasks and demonstrated impressive results in areas such as code generation and test generation, it remains unclear to what extent these models truly understand code execution (Liu et al., 2023). The task of accurately determining which lines of a method are executed based on a given test case and its inputs requires a deep understanding of the underlying code execution dynamics. This motivates the need for a dedicated task, referred to as Code Coverage Prediction, which specifically evaluates the capability of LLMs in comprehending code execution. Further, a model capable of this task is independently useful as it can amortize the expensive code coverage computation process, or function in cases where normal code coverage is not possible to compute.

In this paper we formalize the Code Coverage Prediction task, with the primary objective of evaluating the capability of LLMs in understanding code execution by accurately determining which lines of a method are executed based on a given test case. To facilitate evaluation, we have curated a comprehensive dataset named COVERAGEEVAL, consisting of coverage-annotated methods. This dataset is created by executing tests and code from the HumanEval dataset, allowing us to collect valuable code coverage information. We have organized and made this curated dataset available on GitHub, enabling researchers to explore and advance code coverage prediction techniques and LLM code understanding.

We evaluate the performance of four state-of-the-art LLMs widely employed for code-related tasks: OpenAI's GPT-4 and GPT-3.5, Google's BARD, and Anthropic's Claude. Our ultimate goal is to gain insights into the capabilities of LLMs in predicting code coverage, offering a promising alternative to execution-based coverage measurement in various scenarios. This approach proves advantageous when the costs associated with program building and execution are prohibitive, when code coverage needs to be invoked multiple times, when only code snippets are available (e.g., in server-side scenarios), or when errors in the project prevent complete builds. Additionally, this task introduces a novel metric for assessing code understanding and serves as a valuable (pre-)training objective. By training models to excel in this task, we believe we can enhance their overall performance on code-related tasks.

This paper makes the following contributions:

- *Code Coverage Prediction Task*: We propose a novel task to assess the capability of LLMs in understanding code execution by accurately predicting executed lines of a method based on a given test case and inputs.

- *Evaluation of State-of-the-Art LLMs*: We evaluate four prominent LLMs (GPT-4, GPT-3.5, BARD, and Claude) on the Code Coverage Prediction task, providing insights into their performance and understanding of code execution.

- *Curated Dataset*: We curate a comprehensive dataset (COVERAGEEVAL) of coverage-annotated methods and test cases, derived from the HumanEval dataset. This dataset is openly available on GitHub[1] (Microsoft, 2023) enabling further research and advancement in code coverage prediction techniques.

2 Background

Code coverage is a measure of the degree to which a test suite exercises a software system (Ivanković et al., 2019). Code coverage is commonly computed by means of instrumentation. This technique inserts instrumentation code in various locations within the code or binaries of the program under

[1] https://github.com/microsoft/coverage-eval

test, in order to monitor its execution. This inserted code provides counters to record which function or statement of the program have been executed by the test suite. Inserting these additional statements within the original code leads to execution overhead, which can be significant especially for large software programs (Tikir and Hollingsworth, 2002).

The most common coverage metric is computed at statement level, where statement refers to a syntactic unit of code (*e.g.,* assignment, invocation, assertion), often matching a single line of code. The coverage indicates whether a statement has been executed or not, and aggregated metrics can be computed at function/program level to measure the amount of statements covered by a test suite. In the example in Figure 1, the test case t executes four statements in m, which constitutes $\sim 44\%$ statement coverage for the method m.

Given statement coverage information, other coverage criteria and metrics can be obtained by means of static analysis. Statement coverage information regarding control structure (*e.g.,* `if-else` and `case` statements) can be used to compute branch coverage, which measure how many logical branches in the program have been executed. In the example in Figure 1 only one branch is executed (*i.e.,* `else if (x > 0)`), while the other two branches are missed by the test case t.

In the remainder of this paper we will focus on statement coverage, from which other coverage criteria can be obtained.

3 Code Coverage Prediction Task

Given a method under test (focal method) m, composed of n statements $S_m = s_1, s_2, \ldots, s_n$, and a test case t which exercises the method m, the coverage-annotated focal method $\text{cov}(m, t)$ is composed of a sequence of n statements $S_m^t = s_1^*, s_2^*, \ldots, s_n^*$, where each statement s_i^* represents the coverage-annotated statement of s_i in m. Specifically, s_i^* is marked with one of the three possible coverage symbols $c \in \{>, !, -\}$, where the symbol $>$ identifies statements that have been executed by t, the symbol ! identifies statements that have been missed by t, and the symbol $-$ identifies statements that are unreachable. This defines a sequence of n coverage symbols $C_m^t = c_1, c_2, \ldots, c_n$, where $c_i \in \{>, !, -\}$.

We define the Code Coverage Prediction Task as the problem of predicting the coverage-annotated sequence of statements S_m^t given the focal method m and a test case t. Formally, this problem can be defined in terms of inputs and expected output:

Input

- Focal Method: m
- Test Case: t

Output

- $S_m^t = s_1^*, s_2^*, \ldots, s_n^*$
  or
- $C_m^t = c_1, c_2, \ldots, c_n$

Specifically, the output can be either the coverage-annotated sequence of statements S_m^t, or the sequence of coverage symbols C_m^t, which can then combined with the original sequence of statements $S_m = s_1, s_2, \ldots, s_n$, to obtain the coverage-annotated sequence of statements $S_m^t = s_1^*, s_2^*, \ldots, s_n^*$ comprising the coverage $\text{cov}(m, t)$. This final step is performed by aligning the two sequences and obtaining $s_i^* = c_i + s_i$, where the $+$ operation refers to string concatenation.

Let us take as example the focal method m and test case t in Figure 1. The model is expected to predict either the coverage-annotated sequence of statements S_m^t or the sequence of coverage symbols: `> > ! > > ! ! ! -`.

3.1 Coverage Prediction for Pre-Training

We propose that the code coverage prediction task introduced in our paper can serve as a valuable pre-training task for LLMs focused on code generation. While current pre-training tasks, such as Masked Language Modeling (MLM) help models understand code syntax and semantics by analyzing vast amounts of raw text representing code, our proposed task enables the model to learn about code execution, which is not technically discoverable by source code text alone.

To accomplish this pre-training, we suggest augmenting the training data with extensive coverage logs obtained from Continuous Integration/Continuous Deployment (CI/CD) pipelines. These logs contain valuable information about code coverage from regression tests executed during pull requests or commits.

By exposing the models to these coverage logs during pre-training, they can learn to associate test cases and inputs with the specific lines of code that are executed. This pre-training approach enhances

the models' understanding of how different parts of the code are exercised by various test scenarios. Consequently, the models can acquire a deeper comprehension of the relationships between inputs, tests, and code execution, leading to improved code generation capabilities.

Integrating coverage prediction as a pre-training task could enable models to learn from real-world test scenarios, capturing the nuances of code execution in practical settings. This real-world exposure should enhances the models' ability to generate code that aligns with actual testing practices.

Furthermore, incorporating coverage prediction as a pre-training task opens up possibilities for transfer learning. Models pre-trained on coverage prediction can be fine-tuned on downstream tasks, such as bug detection or test case generation, where understanding code execution is crucial. The models' pre-existing knowledge of code coverage can provide a solid foundation for these related tasks, potentially improving their overall performance.

4 COVERAGEEVAL Dataset

In addition to proposing the code coverage prediction task, this paper also introduces COVERAGEEVAL, a dataset specifically designed for evaluating LLMs on this task. This section outlines the process of curating this dataset, which begins with the HumanEval dataset (Chen et al., 2021). By executing test cases from the HumanEval dataset, we gather code coverage information. To create COVERAGEEVAL, we parse the code coverage logs generated during the execution of the test cases. This parsing step enables us to extract the relevant coverage annotations. We then carefully structure and export the dataset in a format that facilitates its use and evaluation by researchers and practitioners alike.

By curating this dataset, we aim to provide a standardized benchmark for evaluating LLMs on the code coverage prediction task. The availability of COVERAGEEVAL enables researchers to explore and advance code understanding, fostering innovation and enabling the development of more effective models.

4.1 HumanEval

The HumanEval dataset consists of 164 handwritten problems and their code solutions, where each problem is a programming task involving language comprehension, reasoning, algorithms and/or simple mathematics (Chen et al., 2021). Each code solution in the dataset includes a function signature, a docstring containing the problem description, a function body, and several unit tests. We extend the HumanEval dataset to include coverage, calculated using the function body and the respective unit tests.

4.2 Coverage Analysis

In this section, we describe the steps taken to analyze the code coverage on the HumanEval dataset and create our COVERAGEEVAL dataset.

Each code solution in the HumanEval dataset is accompanied by a single test case, which includes multiple asserts designed to test the correctness of the code solution based on the given problem's functional requirements. These asserts cover various inputs, scenarios, and code statements/branches. To enhance the dataset and increase the complexity of each data point, we split the single test case into multiple test cases, each containing a single assert. This splitting process allows us to generate additional method-test pairs, as well as making each data point more challenging. The original test case may cover most of the lines and branches in the method, but each individual assert covers only a subset of them.

By performing this split, we create a more diverse set of method-test pairs within the dataset. Each individual test case invokes the focal method once and covers a subset of the statements and branches within the method. This enables us to evaluate the LLMs' ability to predict code coverage at a more granular level, going beyond the overall coverage of the method. It also adds complexity to the task, as predicting coverage for each assert requires a deeper understanding of the code and its potential execution paths.

Subsequently, we execute the extracted test cases individually with `pytest`. During the execution, we also enable the coverage computation using `coverage.py`. To do so, we run the following command: `coverage run -m pytest <test_name>` where `<test_name>` is each individual test in the dataset.

Next, for each test case t, we analyze the corresponding coverage report obtained by the test execution in order to extract the annotated coverage $\text{cov}(m, t)$. The coverage report marks each source code line in the file with coverage information, specifying whether the statement has been

executed or not.

We automatically parse this report and extract the corresponding annotated coverage $\text{cov}(m, t)$. At the end of this process, we obtained a dataset where each data point is formed by a triplet $d = \{m, t, \text{cov}(m, t)\}$.

4.3 Data Format

The COVERAGEEVAL dataset maintains the structure of the HumanEval dataset, with the addition of coverage information for each test. Each record corresponds to a unique problem and contains the following fields:

- Problem ID: A unique ID for the problem
- Problem: The name of the method written to solve the problem
- Method: The method contents, including a function signature, a docstring with the details of the problem, and the function body.
- Tests: A list of unit tests for the problem. Each item in the list includes the unique ID of the test and the code of the test. We have also added coverage information for each test in the following two forms:
    1. Coverage: The code of the method, with each line annotated with `>`, `!` or `-` for code that is executed, missed or unreachable by the given test.
    2. Coverage Sequence: A list of equal length to the number of lines in the method, where each value in the list is `>`, `!` or `-`, depending on the status of the respective line of code in the method.

Figure 3 (Appendix) shows a sample record from the COVERAGEEVAL dataset. COVERAGEEVAL is available to the public via GitHub (Microsoft, 2023).

Table 1 reports the statistics for the COVERAGEEVAL dataset in terms of number of problems, code solutions, tests, and coverage symbols. The discrepancy between number of problems and solutions is explained by the fact that some problems have multiple solutions. It is also worth noting that while our dataset currently does not contain any unreachable code (-), we have proactively considered the potential presence of unreachable code while designing the task.

Problems	Solutions	Tests	Coverage Symbols		
			Executed (>)	Missed (!)	Unreachable (-)
158	164	1160	20037	1734	0

Table 1: COVERAGEEVAL statistics.

5 Evaluating LLMs

In this section, we present our evaluation of state-of-the-art Language Models (LLMs) for the proposed task of Code Coverage Prediction. We selected four highly regarded LLMs that are not only popular for code generation but also widely used for other Natural Language (NL) tasks. The LLMs we employed for this evaluation are OpenAI's GPT-4 and GPT-3.5, Google's BARD, and Anthropic's Claude.

GPT-3.5 (Brown et al., 2020) and GPT-4 (OpenAI, 2023) are large language models developed by OpenAI which are Transformer-style models (Vaswani et al., 2017) pre-trained to predict the next token in a document. Both models were then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) (Christiano et al., 2017). GPT-4 improves over the predecessor by accepting as input both images and text (multimodal model) and producing text as output. BARD is a conversational AI developed by Google based on LaMDA(Thoppilan et al., 2022) a Transformer-based language models trained on dialogue (Adiwardana et al., 2020). Anthropic Claude is a 52-billion-parameter LLM developed by Anthropic. Claude was pretrained on a large text corpus and finetuned with "RL from AI Feedback" (RLAIF), where AI feedback are steered by a small set of principles drawn from a "constitution" defined by humans (Bai et al., 2022).

5.1 Experimental Design

When evaluating the LLMs on the code coverage prediction task, we designed the experiments to assess their performance on non-trivial coverage sequences while progressively providing more information and examples.

First, we filtered out data points $d = \{m, t, \text{cov}(m, t)\}$ where the coverage sequence is *trivial* consisting exclusively of the symbol `>`. These cases represent methods with no branches or where the test case covers every statement in the focal method. Although these data points are included in the COVERAGEEVAL dataset, we excluded them from this specific evaluation. The subset of data points containing only trivial symbols is

reported in our online appendix. It's important to note that no data points in the dataset has a coverage sequence consisting solely of ! or - symbols. After this filtering step, we were left with 478 data points on which we evaluated the LLMs.

The prompt used to evaluate the LLMs was designed to include the following sections:

- System NL prompt: a prompt providing a natural language description of the task, aimed at conveying the task to the LLM.
- Examples: zero, one, or multiple examples of the task.
- Focal Method m and Test Case t.

In terms of the System NL prompt, our evaluation involved experimenting with various prompts and descriptions. We achieved the most favorable outcomes by utilizing a system prompt that emulates a terminal environment (e.g., python terminal). Within this prompt, we instructed the LLM to generate the code coverage output based on a given test case and method. For OpenAI models, we included this prompt in the specific system prompt section, while for BARD and Claude, we incorporated it as the initial part of the prompt.

To comprehensively assess the LLMs' performance, we conducted evaluations using different numbers of examples for the code coverage prediction task. Specifically, we employed zero-shot, one-shot, and multi-shot prompting approaches. This allowed us to examine the impact of example availability on the models' performance and their ability to generalize the task across various methods.

When selecting examples for evaluating coverage on a particular method m_i, we took care to prevent data leakage and encourage the LLMs to generalize their predictions to other methods. To achieve this, we randomly sampled a data point $\{m_j, t, \text{cov}(m, t)\}$ where $m_j \neq m_i$ when providing examples.

Finally, the prompt provides a focal method m and a corresponding test case t for which we expected the model to predict the code coverage. Figure 2 shows an example of the prompt we designed.

Inference is performed on all the LLMs with temperature and topp set to 0, and generating one sample.

System NL Prompt

```
You are a terminal. Instruction:
When user runs:
coverage run -m  pytest code.py

then you'll cat the file code.py,
with each line starting with either of the two symbols below:

> if the line is executed
! is the line is not executed

Example output:
> line1
! line2
> line3
...
> linen

You job is to figure out which line will be executed
given different test cases.
```

Examples

```
(anaconda3-2020.11) cat code.py
def split_words(txt):
    ...

(anaconda3-2020.11) cat test.py
def test():
    assert split_words("Hello,world!") == ["Hello","world!"]
    assert True

(anaconda3-2020.11) coverage run -m  pytest test.py
> def split_words(txt):
>     if " " in txt:
!         return txt.split()
>     elif "," in txt:
>         return txt.replace(',',' ').split()
!     else:
    ...
```

Focal Method m + Test Case t

```
(anaconda3-2020.11) cat code.py
def <focal_method>
    ...

(anaconda3-2020.11) cat test.py
def test():
    ...

(anaconda3-2020.11) coverage run -m  pytest test.py
```

Figure 2: Code Coverage Prediction Task Prompt: (i) System NL Prompt instruct the LLM to operate as in a terminal environment; (ii) zero, one, or multiple examples of the coverage prediction task may be shown; (iii) the current focal method m and test case t are provided

5.2 Evaluation Metrics

In this section we describe the evaluation metrics.

Given the method m, the test case t, and the sequence of coverage symbols $C_m^t = c_1, c_2, \ldots, c_n$, where $c_i \in \{>, !, -\}$, the model generates a predicted sequence of coverage symbols $\hat{C}_m^t = \hat{c}_1, \hat{c}_2, \ldots, \hat{c}_n$. We consider the following metrics to evaluate the performances of our proposed approach.

Model	zero-shot			one-shot			multi-shot		
	Match	Stmt	Branch	Match	Stmt	Branch	Match	Stmt	Branch
OpenAI GPT-4 (gpt-4)	**25.75**	**84.47**	**20.16**	**22.85**	**90.71**	**22.65**	**30.04**	**90.5**	**22.5**
OpenAI GPT-3.5 (gpt-3.5-turbo)	0	39.87	8.33	8.17	76.53	17.17	11.03	82.29	17.9
Google BARD (text-bison-001)	0	81.27	17.21	1.87	86.93	19.63	21.56	85.66	20.52
Anthropic Claude (claude-1.3)	3.9	84.47	20.07	4.83	83.21	19.16	6.88	55.7	12.23

Table 2: LLMs performances on the Code Coverage Prediction Task. The table reports the percentages of predicted coverage sequences that match the ground truth (Match), the percentage of correct coverage symbols for statements (Stmt), and specifically for branches (Branch). Evaluation performed for zero-shot, one-shot, and multi-shot.

5.2.1 Perfect Sequence Match

The perfect sequence match metric counts the number of times that the predicted sequence $\hat{C}_m^t$ exactly matches (symbol-by-symbol) the target coverage sequence C_m^t. This represents the case where the model predicts the coverage with perfect accuracy for all the statements and branches.

5.2.2 Statement Correctness

The statement correctness metric measures the percentage of statements for which the execution prediction is correct. This is equivalent to the percentage of symbols in the predicted sequence that match the target sequence.

5.2.3 Branch Correctness

The branch correctness metric measures the percentage of branch-specific statements for which the execution prediction is correct. The branch correctness only considers the symbols associated with branch statements. It measures the percentage of symbols in the predicted sequence (associated with branches) that match the symbols in the target sequence.

6 Results

Table 2 presents the performance of different LLMs on the Code Coverage Prediction task. The table showcases the percentage of predicted coverage sequences that match the ground trught (Match), the percentage of correct coverage symbols for all the statements (Stmt), and the percentage of correct coverage symbols when only considering branch statements (Branch). Evaluation performances are computed using zero-shot, one-shot, and multi-shot prompting.

OpenAI GPT-4 demonstrates the highest performance on this task, achieving 24.75% exact match with zero-shot prompting and improving to 30% with multi-shot prompting, where up to 6 examples are provided in the prompt. Notably, the other LLMs achieve low exact matches with zero-shot prompting (between 0 and 4%), suggesting that these foundational models may not have been exposed to coverage logs during their training or that. The second best-performing model is Google BARD, with an exact sequence match reaching 21.5% with multi-shot prompting.

Regarding the percentage of correct coverage statements (see Stmt), most models demonstrate improvement as more examples are included in the prompt. OpenAI GPT-4 obtain the overall best scores between 84% and 90% of statement correctness.

When considering only statements involved in branches (*e.g.,* `if-else`, `while`), it becomes evident that there is a significant drop in correct predictions. In fact, the best performing model, OpenAI GPT-4, accurately predicts a modest 22% of these symbols when one- and multi-shot is used for prompting. It is important to note that this subset of statements, which are intricately connected to branches, presents a greater challenge for evaluation because the LLM must reason about the boolean conditions that determine which branch is covered. Consequently, accurately predicting coverage symbols within this context requires the model to possess a profound understanding of the conditional logic that guides program execution.

Despite the surprisingly strong results of OpenAI GPT-4 on the Code Coverage Prediction task, it should be noted that the model still fails to generate the correct coverage for more than 70% of the method-test pairs in the COVERAGEEVAL dataset. This emphasizes that LLMs have a long way to go in developing a deep understanding of code execution.

We believe that in order to enhance code generation results, these LLMs should gain a comprehensive understanding of code execution under

different inputs and test cases. Therefore, we assert that our dataset and proposed task can contribute to the advancement of LLMs towards this goal.

7 Discussion& Applications

LLMs trained to excel on the Code Coverage Prediction task could offer a promising alternative to traditional execution-based code coverage measurement in various scenarios. In this section, we discuss several use case scenarios where this approach can be valuable and beneficial.

7.1 Expensive Build & Execution

For large software projects with millions of lines of code and numerous dependencies, the build and execution process can be time-consuming and expensive. In such cases, developers may want to analyze the code coverage obtained by newly written tests without waiting for the lengthy build phase. By leveraging LLMs trained on the Code Coverage Prediction task, developers can predict the coverage obtained by the new tests on existing methods without the need to build the entire project or execute the tests. This enables developers to quickly assess whether additional tests are required to cover missed lines or branches in the methods, saving valuable time and resources.

7.2 Limited Code Availability

Traditional code coverage computation requires the complete source code of the codebase to be available for instrumentation and execution. However, there are scenarios where only a partial view of the code is accessible, making code coverage computation impossible using traditional methods.

In cases where limited code availability poses a challenge, the Code Coverage Prediction approach can be employed. For example, when utilizing an AI code generation service from an IDE, developers may transmit only a partial view of the code to the server where the AI model resides. In this scenario, the server can use the proposed approach to predict the code coverage of the AI-generated test cases on the given method. This enables estimation of the code coverage without the need for the entire codebase, addressing privacy concerns and network limitations. The predicted code coverage can then be used to make informed decisions, such as generating additional tests if coverage is insufficient or transmitting the generated tests to the user if coverage is satisfactory.

7.3 Live Coverage

Live Unit Testing, integrated into various IDEs, allows developers to receive real-time feedback on the impact of code changes on existing tests and identifies whether newly added or modified code is covered by existing tests. In this scenario, the Code Coverage Prediction approach can be applied by replacing the actual execution of test cases with an AI inference call to predict the coverage on the modified or newly added methods. This provides developers with immediate feedback on code coverage without the need for executing the entire test suite. By utilizing LLM-based models for code coverage prediction, developers can streamline the testing process and receive timely insights into the coverage of their code changes.

8 Conclusion

In this paper, we introduced the novel task of Code Coverage Prediction, which aims to assess the capabilities of Large Language Models (LLMs) in understanding code execution by accurately predicting the lines of code that are executed based on given test cases. We curated a comprehensive dataset named COVERAGEEVAL, consisting of coverage-annotated methods derived from the HumanEval dataset. This dataset enables researchers to explore and advance code coverage prediction techniques and LLM code understanding.

We evaluated the performance of four state-of-the-art LLMs, namely OpenAI's GPT-4 and GPT-3.5, Google's BARD, and Anthropic's Claude, on the Code Coverage Prediction task. The results demonstrated that GPT-4 achieved the highest performance, with 10.46% exact match with zero-shot prompting and 24.48% with multi-shot prompting. However, none of the models, including GPT-4, achieved high accuracy in predicting code coverage, indicating that LLMs still have a long way to go in developing a deep understanding of code execution.

The Code Coverage Prediction task serves as a valuable metric for assessing code understanding and can potentially contribute to the enhancement of LLMs' overall performance on code-related tasks. By training models to excel in this task, we can improve their ability to comprehend code execution dynamics, which is crucial for tasks such as code generation and test generation.

References

Daniel Adiwardana, Minh-Thang Luong, David R So, Jamie Hall, Noah Fiedel, Romal Thoppilan, Zi Yang, Apoorv Kulshreshtha, Gaurav Nemade, Yifeng Lu, et al. 2020. Towards a human-like open-domain chatbot. *arXiv preprint arXiv:2001.09977*.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code.

Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30.

Marko Ivanković, Goran Petrović, René Just, and Gordon Fraser. 2019. Code coverage at google. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 955–963.

Leslie A Johnson. 1998. Do-178b. *Software Considerations in Airborne Systems and Equipment Certification, Crosstalk Magazine*.

Chenxiao Liu, Shuai Lu, Weizhu Chen, Daxin Jiang, Alexey Svyatkovskiy, Shengyu Fu, Neel Sundaresan, and Nan Duan. 2023. Code execution with pre-trained language models. *arXiv preprint arXiv:2305.05383*.

Microsoft. 2023. Coverage-eval. https://github.com/microsoft/coverage-eval.

OpenAI. 2023. Gpt-4 technical report.

Rob Palin, David Ward, Ibrahim Habli, and Roger Rivett. 2011. Iso 26262 safety cases: Compliance and assurance.

Leanna Rierson. 2017. *Developing safety-critical software: a practical guide for aviation software and DO-178C compliance*. CRC Press.

Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*.

Mustafa M Tikir and Jeffrey K Hollingsworth. 2002. Efficient instrumentation for code coverage testing. *ACM SIGSOFT Software Engineering Notes*, 27(4):86–96.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008.

A CoverageEval Example

Problem: rounded_avg

ID = 104

```
def rounded_avg(n, m):
    """SYNTH You are given two positive integers n and m,
    and your task is to compute the average of the integers
    from n through m (including n and m).
    Round the answer to the nearest integer
    and convert that to binary.
    If n is greater than m, return -1.
    Example:
    rounded_avg(1, 5) => "0b11"
    rounded_avg(7, 5) => -1
    rounded_avg(10, 20) => "0b1111"
    rounded_avg(20, 33) => "0b11010"
    """
    if m < n:
        return -1
    summation = 0
    for i in range(n, m+1):
        summation += i
    return bin(round(summation/(m - n + 1)))
```

Test Cases

```
def test_658():
    assert rounded_avg(185,546) == "0b101101110"
    assert True

def test_659():
    assert rounded_avg(362,496) == "0b110101101"
    assert True

def test_660():
    assert rounded_avg(560,851) == "0b1011000010"
    assert True
```

Coverage-Annotated Method $\{test_id = 660\}$

```
> def rounded_avg(n, m):
>    if m < n:
!        return -1
>     summation = 0
>     for i in range(n, m+1):
>         summation += i
>     return bin(round(summation/(m - n + 1)))
```

Coverage Sequence $\{test_id = 660\}$

[>, >, !, >, >, >, >]

Figure 3: Example record from the CoverageEval dataset. This record is for the rounded_avg problem. We have shown 3 of the unit tests, as well as sample coverage annotation data from one unit test. where > represents executed statements, ! missing statements, and - unreachable code.

B Deployed Systems

We deploy our approach in two systems covering some of the use cases described in the paper.

B.1 System A - Live Coverage

Figure 4 shows the deployment of System A, which provides live coverage prediction for developers directly into their IDE. System A supports the scenario where a developer is writing tests for a given method (*e.g.,* `Fibonacci(n)`) in their codebase. System A provides live coverage information (bottom of Figure 4) where lines covered by the tests are marked with > and highlighted in green and the line missed are marked with ! and highlighted in red.

The benefits provided by System A are the following: (i) no need to build the entire codebase; (ii) no need to execute the tests; (iii) live and lightweight coverage prediction.

B.2 System B - Test Generation with Coverage

Figure 5 shows the deployment of System B, which provides Test Suites with a coverage guarantee. System B supports the scenario where a developer is requesting test cases for a given method and would like to obtain a certain degree of coverage on the method under test. Once the method is transmitted to the Test Generation Service, the Test Generation Model (*i.e.,* an AI-based test generation tool or any other tool) outputs a first batch of test case candidates. The Coverage Prediction Model analyzes these tests and the method under test, and predicts the coverage that these tests achieve on the method. If the coverage is satisfactory (w.r.t. a given criteria and threshold) the tests are transmitted to the IDE and shown to the developer. If the tests do not meet the criteria in terms of coverage, the Test Generation Service requests additional tests from the Test Generation Model (optionally, providing the specific lines/branches which still need to be covered).

The benefits provided by System B are the following: (i) automated test generation with coverage guarantees; (ii) lightweight generation without need of build and test execution on the user side.

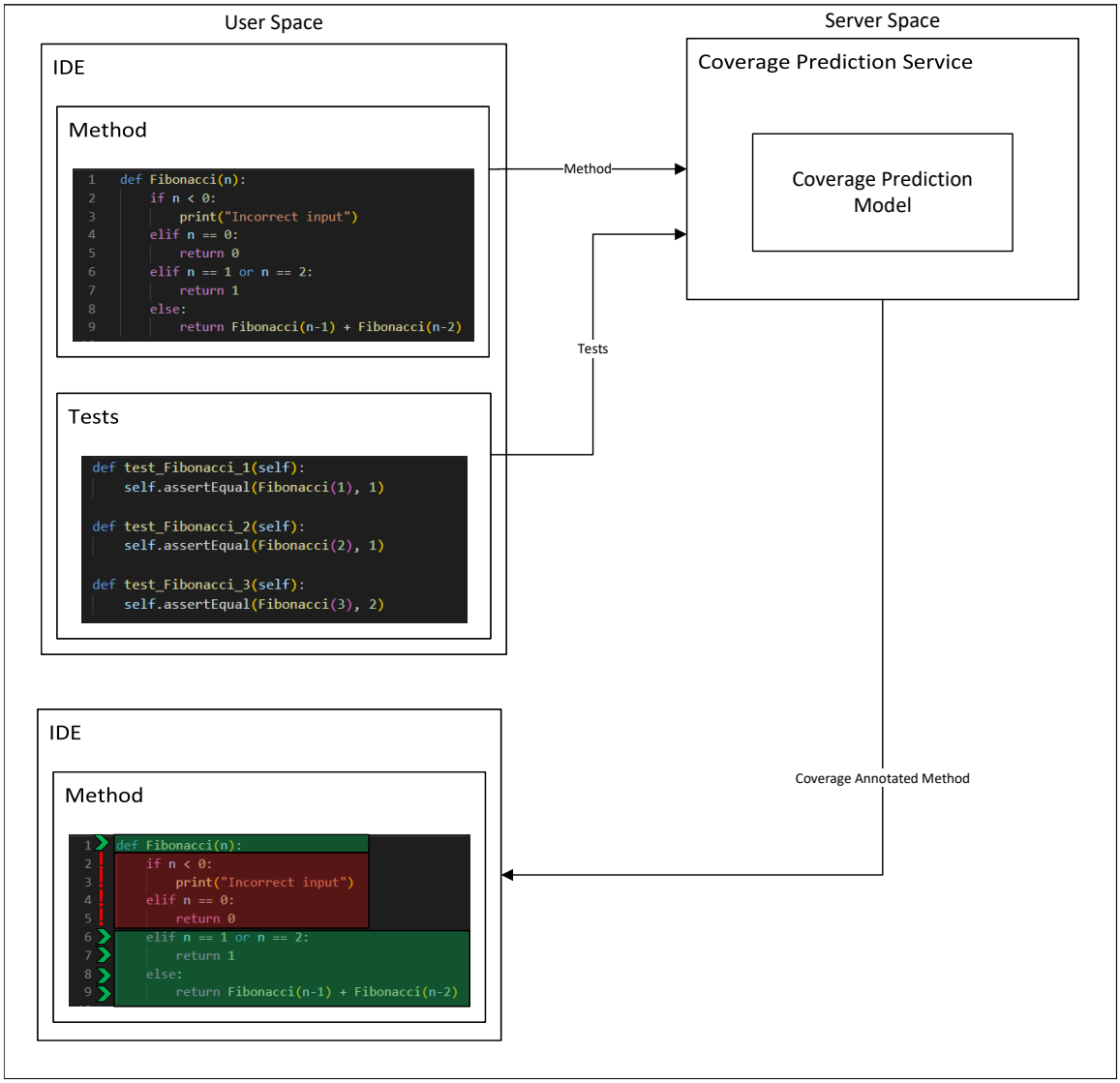

Figure 4: System A - Live Coverage

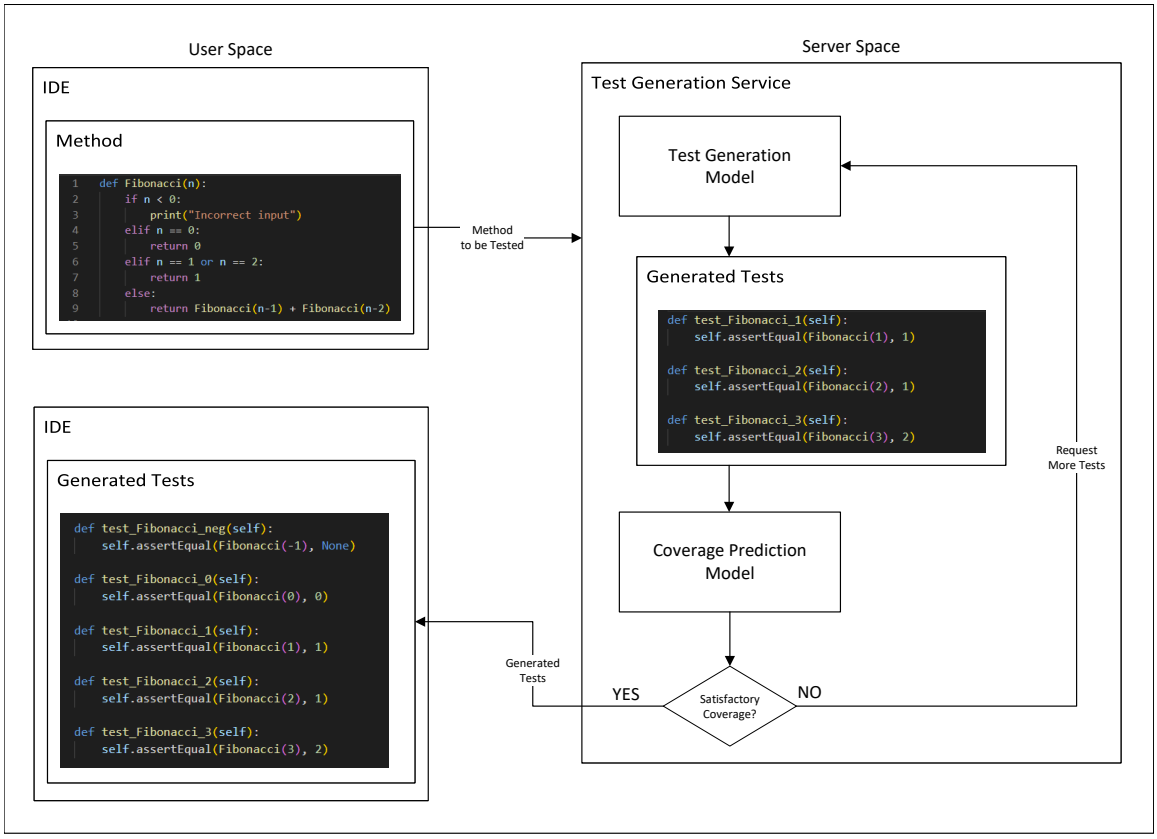

Figure 5: System B - Test Generation with Coverage